NILEBRIDGE PROFESSIONAL ACADEMY

Full Stack Software Development

DEMONSTRATION CATALOGUE · 2026–2027 · CATALOGUE CODE FS

CATALOGUE OVERVIEW

A project-based catalogue covering responsive web foundations, front-end application architecture, backend APIs, relational persistence, testing, and deployment documentation.

ACADEMY AND DATA NOTICE

NILEBRIDGE PROFESSIONAL ACADEMY is a fictional demonstration institution. All programme records, durations, curriculum details, assessments, and career descriptions are synthetic examples created for RAG ingestion and retrieval testing. They are not real academy offerings, verified schedules, official accreditation claims, or employment guarantees.

Programme Directory

The programme records below use stable programme identifiers to support retrieval and metadata validation.

Programme ID	Programme title	Level	Duration
FS-WEB-101	Web Foundations & JavaScript	Foundation	60 guided learning hours
FS-API-301	Backend APIs, Databases & Deployment	Intermediate	100 guided learning hours

Information included for each programme

- Programme identity, level, duration, delivery format, and intended audience
- Entry requirements and programme purpose
- Learning outcomes and week-by-week curriculum
- Practical labs, assessment weights, and grading criteria
- Capstone brief, deliverables, tools, and FAQ
- Provenance and accreditation status

PROGRAMME 01 · FS-WEB-101

Web Foundations & JavaScript

Indicative hours	Duration	Level	Delivery
60 guided learning hours	8 weeks	Foundation	Instructor-led coding workshops and project reviews.

Programme summary

Introduces HTML, CSS, responsive layouts, JavaScript interaction, browser debugging, and accessible web practices.

Intended audience

Beginners learning to build browser-based applications.

Entry requirements

No prior coding required.

Learning outcomes

- Create semantic HTML pages with clear structure.
- Build responsive layouts using CSS fundamentals.
- Implement JavaScript interactions and form validation.
- Debug common browser issues and use Git for version control.

Week-by-week curriculum

Week	Topic	Coverage	Practice / evidence
1	Web and HTML	Browser/server roles, URLs, HTTP overview, semantic structure.	Build a structured content page.
2	CSS fundamentals	Selectors, cascade, box model, spacing, typography.	Style a consistent page.
3	Layout and responsiveness	Flexbox, Grid, breakpoints, and fluid sizing.	Build a responsive landing page.
4	JavaScript basics	Types, functions, arrays, objects, and modules.	Write small reusable functions.
5	DOM and events	Events, forms, validation, and user feedback.	Build an interactive form.
6	Asynchronous requests	Fetch, promises, loading states, and basic error handling.	Display data from a sample endpoint.
7	Accessibility and debugging	Keyboard checks, semantic controls, browser tools, and Git.	Audit and fix issues.
8	Capstone	Combine layout, interaction, and documentation.	Present the site and test checklist.

Practical labs

- HTML structure review
- Responsive CSS exercises
- DOM event implementation
- Browser debugging task

Assessment structure

Component	Weight	Evidence / criteria
Coding labs	30%	Working behavior and readable implementation.
Responsive site	30%	Layout, content structure, and accessibility checks.
Capstone	40%	Completeness, testing, and documentation.

Capstone project

Build a responsive website with multiple content sections, a validated form, a clear navigation pattern, and documented browser checks.

Suggested grading rubric

- Structure and accessibility: 25%
- Responsive layout: 25%
- Interaction correctness: 25%
- Documentation and testing: 25%

Tools and technologies

HTML · CSS · JavaScript · browser developer tools · Git

Frequently asked questions

Do I need React for this course?

No. The focus is on browser fundamentals before introducing a framework.

What does responsive mean?

The layout adapts to different screen sizes and remains usable across supported viewports.

Is a website finished when it looks correct?

No. Interaction, keyboard access, validation, error states, and testing also matter.

Record and provenance

Department: Software Development | Skill level: Foundation | Accreditation: not verified | Country: not specified | Source status: synthetic demonstration data | Catalogue: Full Stack Software Development | Programme: Web Foundations & JavaScript | Source file: 03_Full_Stack_Expanded_Catalogue.pdf.

PROGRAMME 02 · FS-API-301

Backend APIs, Databases & Deployment

Indicative hours	Duration	Level	Delivery
100 guided learning hours	10 weeks	Intermediate	Practical API labs, code review, and integrated project work.

Programme summary

Covers REST-style API design, request validation, database persistence, authentication concepts, automated testing, configuration, and deployment documentation.

Intended audience

Learners with basic programming knowledge who want to build service-backed applications.

Entry requirements

Programming fundamentals and introductory SQL concepts.

Learning outcomes

- Design endpoints with consistent HTTP methods and status codes.
- Separate routing, business logic, and database access.
- Model relational data and implement CRUD operations.
- Validate input, handle errors, and test important behavior.
- Document environment configuration and repeatable startup steps.

Week-by-week curriculum

Week	Topic	Coverage	Practice / evidence
1	HTTP and API design	Methods, status codes, resource naming, and request/response shape.	Design an endpoint contract.
2	Service architecture	Routes, controllers, services, and separation of responsibilities.	Refactor a small API.
3	Validation and errors	Input schemas, predictable errors, and safe messages.	Test invalid requests.
4	Relational schema	Keys, constraints, relationships, and migrations.	Design a small schema.
5	CRUD and transactions	Create, read, update, delete, and data-integrity basics.	Implement persistent endpoints.
6	Authentication concepts	Identity, roles, authorization, and safe secret handling.	Document access rules.
7	Automated testing	Unit and integration tests, fixtures, and edge cases.	Test critical API paths.
8	API documentation	Document endpoints, schemas, and example requests.	Publish an API specification.
9	Configuration and deployment	Environment variables, logs, build steps, and deployment concepts.	Create a startup runbook.
10	Capstone review	Integrate client, API, and database; review reliability.	Demonstrate the application.

Practical labs

- REST contract design
- Database schema and CRUD
- API error testing
- Integration test suite

Assessment structure

Component	Weight	Evidence / criteria
API labs	25%	Endpoint design and validation.
Database task	20%	Schema integrity and persistence.

Component	Weight	Evidence / criteria
Testing and documentation	20%	Coverage of key flows and usable instructions.
Capstone	35%	Integrated, tested, documented service.

Capstone project

Deliver a service with documented endpoints, a relational database, input validation, automated tests, and clear setup instructions. Backend framework is selected and recorded at project kickoff.

Suggested grading rubric

- API design: 20%
- Data integrity: 20%
- Validation and testing: 25%
- Documentation and integration: 35%

Tools and technologies

One selected backend stack: .NET/C# or Node.js · REST · SQL database · API testing tool · Git

Frequently asked questions

Does REST require a specific language?

No. REST-style API concepts can be implemented in many backend languages.

Should database errors be returned directly to users?

No. Responses should be useful without exposing credentials, internal queries, or sensitive implementation details.

What is a migration?

A versioned change to the database schema that can be applied in a controlled sequence.

Record and provenance

Department: Software Development | Skill level: Intermediate | Accreditation: not verified | Country: not specified | Source status: synthetic demonstration data | Catalogue: Full Stack Software Development | Programme: Backend APIs, Databases & Deployment | Source file: 03_Full_Stack_Expanded_Catalogue.pdf.